

Cloud Services Architecture

Dr. Animesh Chaturvedi

Assistant Professor: IIIT Dharwad

Young Researcher: Pingala Interaction in Computing

Young Researcher: Heidelberg Laureate Forum

Postdoc: King's College London & The Alan Turing Institute

PhD: IIT Indore MTech: IIITDM Jabalpur



Indian Institute of Technology Indore
भारतीय प्रौद्योगिकी संस्थान इंदौर



PDPM

Indian Institute of Information Technology,
Design and Manufacturing, Jabalpur

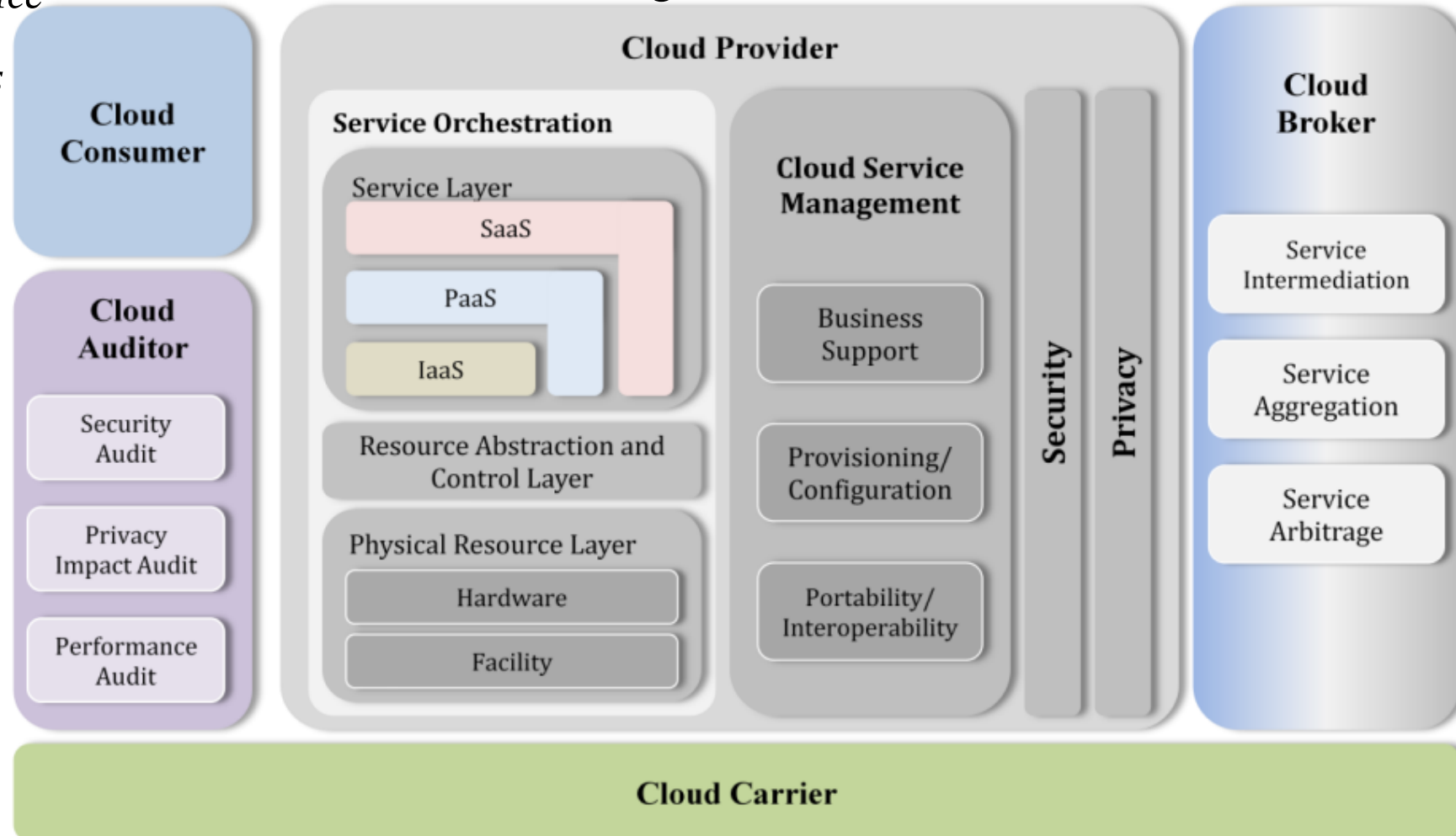
NIST Cloud Computing Architecture

Cloud Service Characteristics

- *On demand self-service*
- *Broad network access*
- *Resource pooling*
- *Rapid elasticity*
- *Measured service*

Understanding of the requirements, uses, characteristics and standards of cloud computing.

Cloud high-level architecture



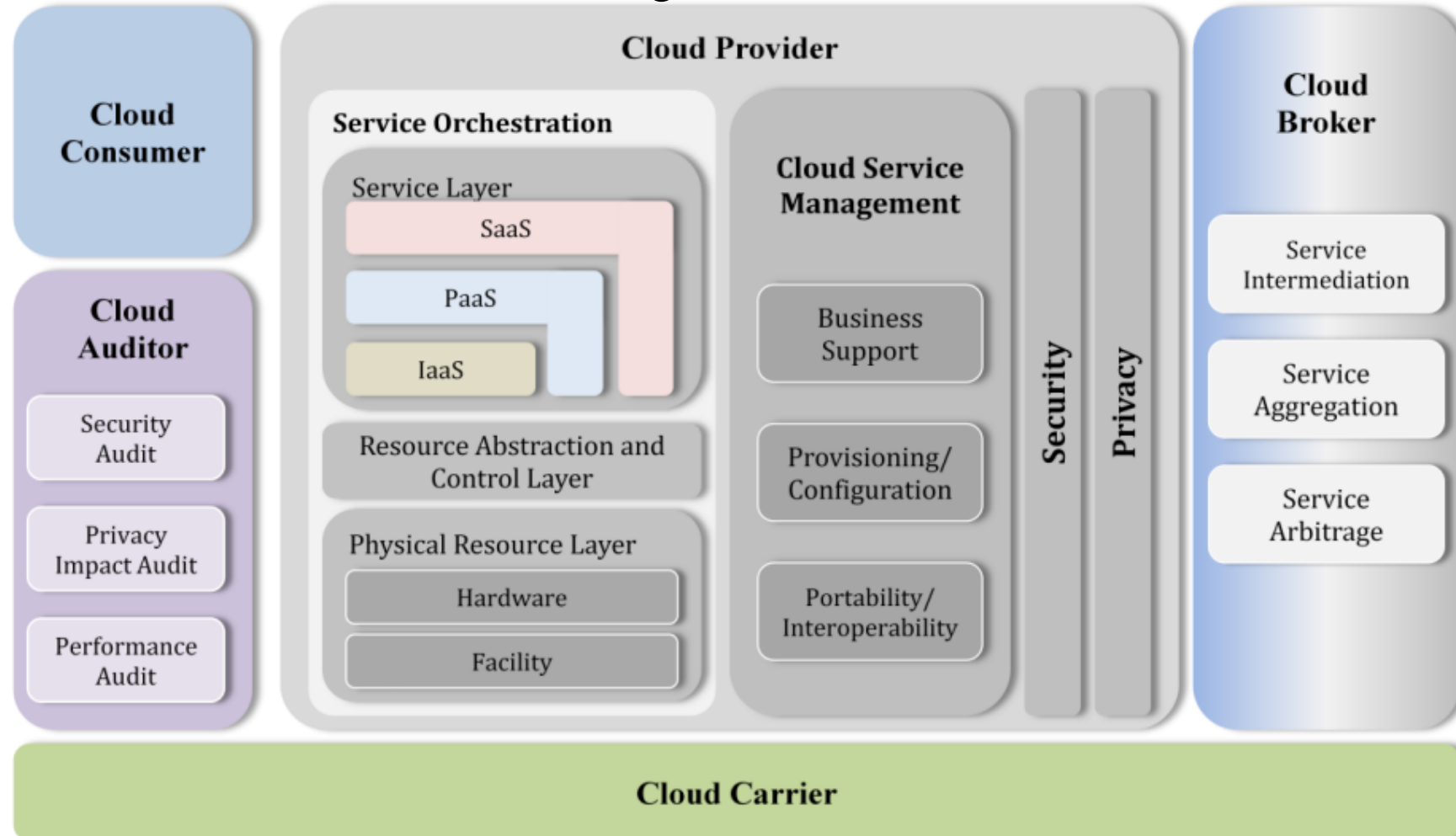
Cloud Conceptual Reference Model

Five Cloud Actors

1. *Cloud Consumer*
2. *Cloud Provider*
3. *Cloud Broker*
4. *Cloud Auditor*
5. *Cloud Carrier*

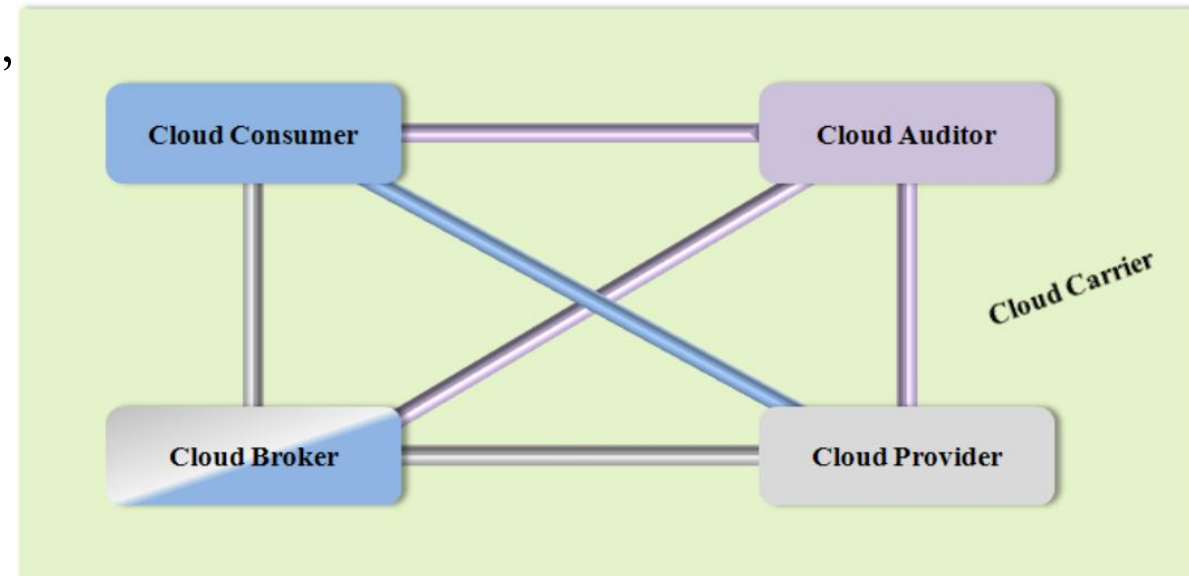
Actors with their roles, responsibilities, activities and functions in cloud computing.

Cloud high-level architecture



Actors in Cloud Computing

- **Cloud Consumer** A person or organization that maintains a business relationship with, and uses service from, *Cloud Providers*.
- **Cloud Provider** A person, organization, or entity responsible for making a service available to interested parties.
- **Cloud Auditor** A party that can conduct independent assessment of cloud services, information system operations, performance and security of the cloud implementation.
- **Cloud Broker** An entity that manages the use, performance and delivery of cloud services, and negotiates relationships between *Cloud Providers* and *Cloud Consumers*.
- **Cloud Carrier** An intermediary that provides connectivity and transport of cloud services from *Cloud Providers* to *Cloud Consumers*.

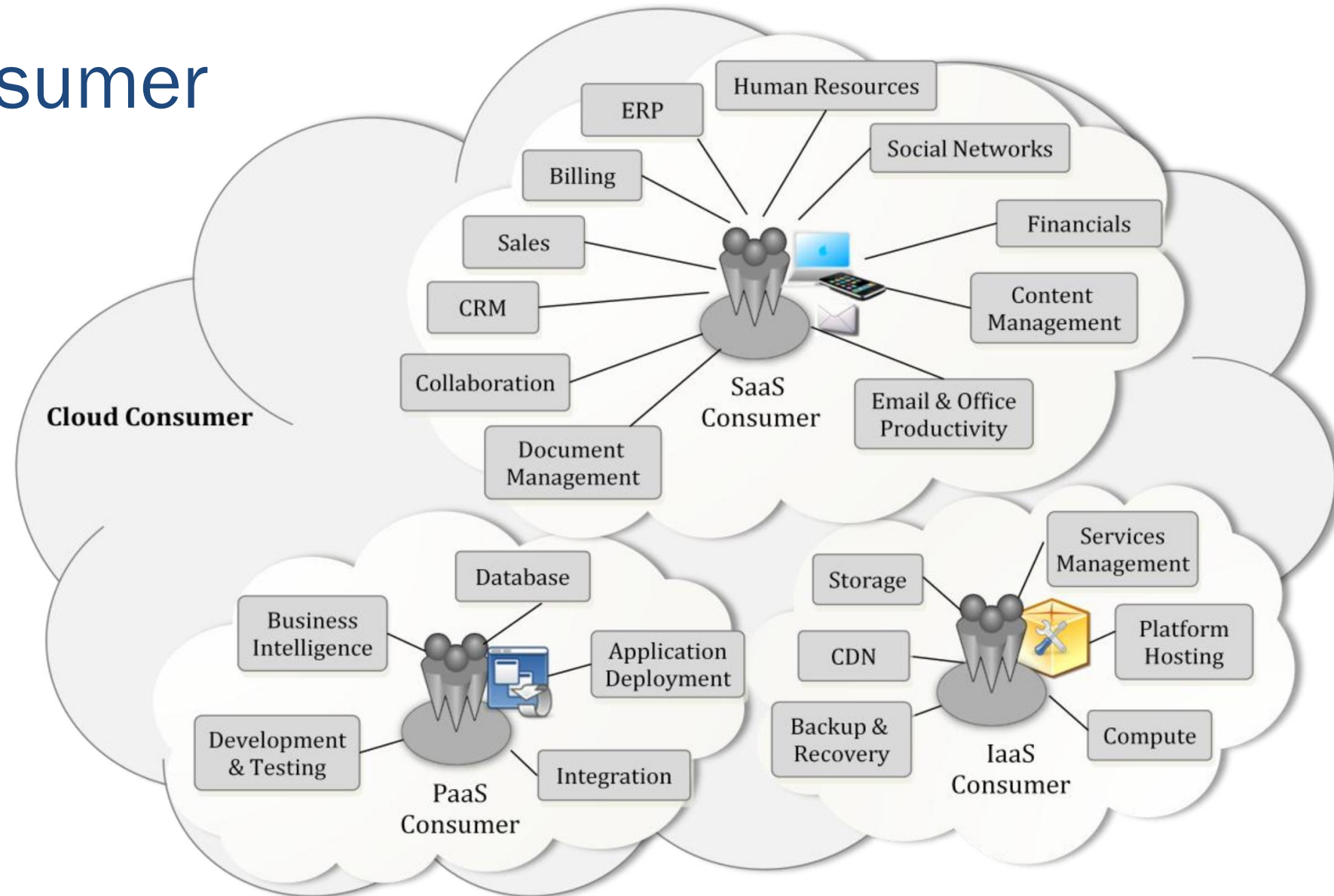


Cloud Consumer

- Cloud consumer browses & uses the service.
- Cloud consumer sets up contracts with the cloud provider.
- Cloud consumers need SLAs to specify the technical performance requirements fulfilled by a cloud provider.
- SLAs cover the quality of service, security, remedies for performance failures.
- A cloud provider list some SLAs that limit and obligate the cloud consumers by must acceptance.
- Cloud consumer can freely choose a cloud provider with better pricing with favorable conditions.
- Pricing policy and SLAs are non-negotiable.

Cloud Consumer

- SaaS Consumer
- PaaS Consumer
- IaaS Consumer



Cloud Consumer

- SaaS consumers can be organizations that provide their members with access to software applications, end users who directly use software applications, or software application administrators who configure applications for end users.
- PaaS consumers can be application developers or administrators
 1. who design and implement application software
 2. application testers who run and test applications
 3. who publish applications into the cloud
 4. who configure and monitor application performance.
- IaaS consumer can be system developers, system administrators and IT managers who are interested in creating, installing, managing and monitoring services for IT infrastructure operations.

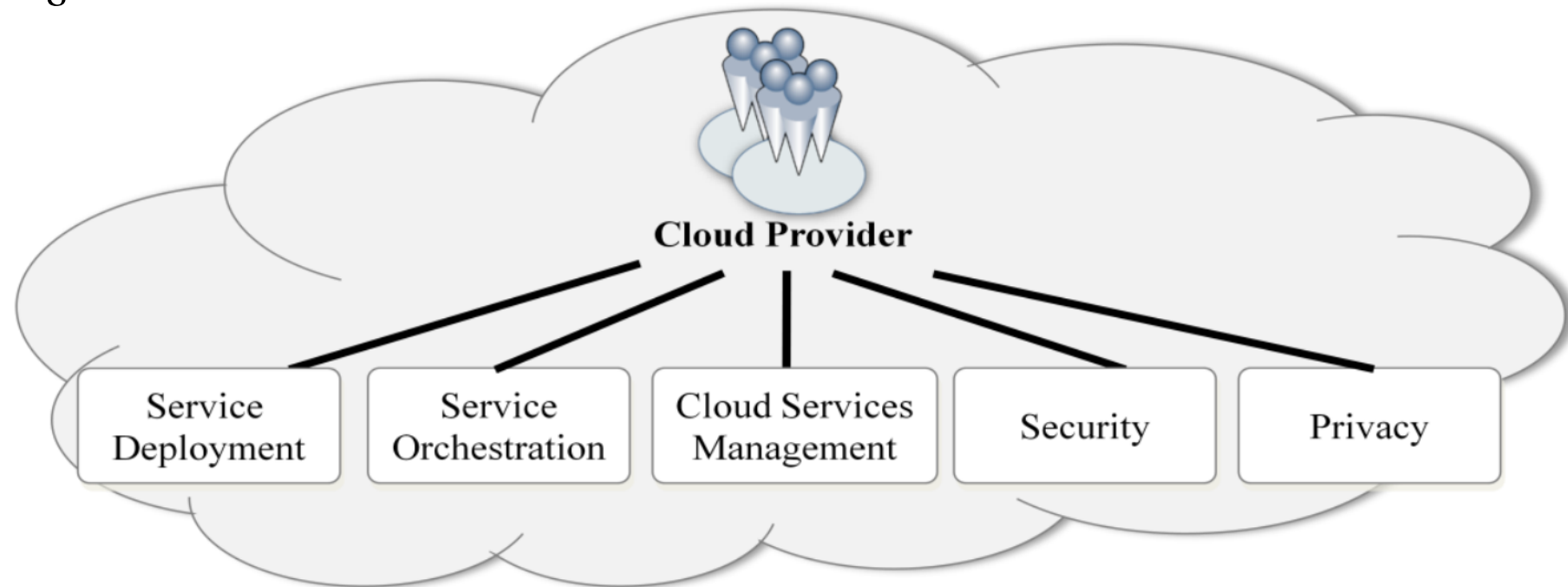
Cloud Consumer Billing

- SaaS consumers can be **billed** based on the number of end users, the time of use, the network bandwidth consumed, the amount of data stored or duration of stored data.
- PaaS consumers can be **billed** according to, processing, database storage and network resources consumed by the PaaS application, and the duration of the platform usage.
- IaaS consumer can be **billed** according to the amount or duration of the resources consumed, such as CPU hours used by virtual computers, volume and duration of data stored, network bandwidth consumed, number of IP addresses used for certain intervals.

Cloud Provider

Five major activities of Cloud Provider's

- *Service deployment*
- *Service orchestration*
- *Cloud service management*
- *Security*
- *Privacy*



NIST Cloud Computing Reference Architecture

Cloud Provider

- Cloud Provider acquires and manages the computing infrastructure required for providing the services, runs the cloud software that provides the services, and makes arrangement to deliver the cloud services to the Cloud Consumers through network access.
- SaaS provider deploys, configures, maintains and updates the operation of the software applications on a cloud infrastructure. SaaS provider maintains the expected service levels to cloud consumers.
- PaaS Provider manages the computing infrastructure for the platform and components (runtime software execution stack, databases, and other middleware).
- IaaS Cloud Provider provides physical hardware and cloud software that makes the provisioning of these infrastructure services, for example, the physical servers, network equipments, storage devices, host OS and hypervisors for virtualization.

Cloud Broker

- Integration of cloud services can be complex for consumers. Hence cloud broker, is needed.
- Broker manages the use, performance and delivery of cloud services and negotiates relationships between cloud providers and cloud consumers.
- In general, a cloud broker can provide services in three categories:
 - *Service Intermediation*: Broker enhances a service by improving capability and providing value-added services to consumers. The improvement can be managing access to cloud services, identity management, performance reporting, enhanced security, etc.
 - *Service Aggregation*: Broker combines and integrates multiple services into one or more new services. The broker provides data integration and ensures the secure data movement.
 - *Service Arbitrage*: It is similar to service aggregation with the flexibility to choose services from multiple agencies. For example, broker can select service with the best response time.

Scenarios in Cloud: 1

1. Cloud consumer interacts with the cloud broker instead of contacting a cloud provider directly.
2. The cloud broker may create a new service (mash up) by combining multiple services or by enhancing an existing service.
3. Actual cloud providers are invisible to the cloud consumer.

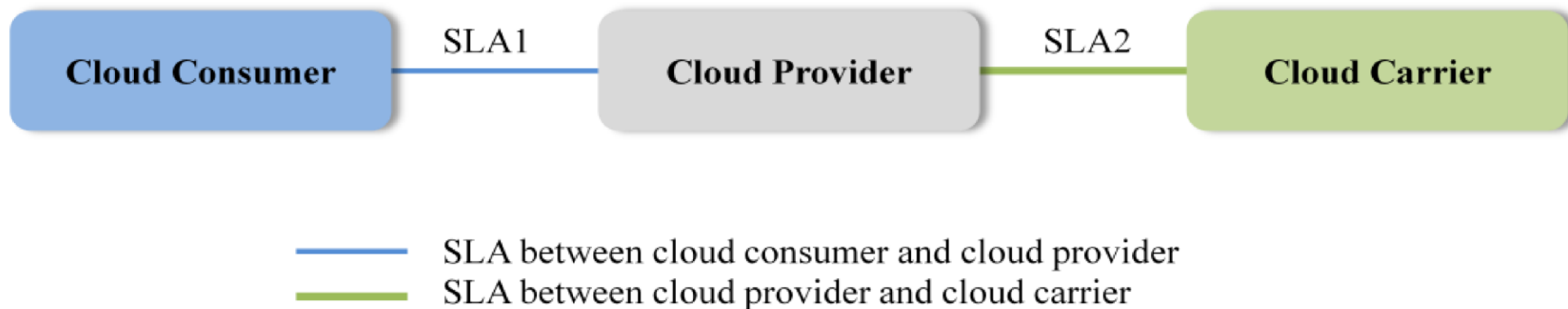


Cloud Carrier

- Cloud carriers provide access to consumers through network, telecommunication and other access devices.
- For example, cloud consumers can obtain cloud services through network access devices, such as computers, laptops, mobile phones, mobile internet devices (MIDs), etc.
- The distribution of cloud services is normally provided by network and telecommunication carriers or a *transport agent*, where a transport agent refers to a business organization that provides physical transport of storage media such as high-capacity hard drives.
- Cloud provider can set up SLAs with a cloud carrier to provide services consistent with the level of SLAs offered to cloud consumers.

Scenarios in Cloud: 2

1. Cloud carriers provide the connectivity and transport of cloud services from cloud providers to cloud consumers.
2. Cloud provider participates in and arranges for two unique service level agreements (SLAs), one with a cloud carrier (e.g. SLA2) and one with a cloud consumer (e.g. SLA1).
3. A cloud provider may request cloud carrier to provide dedicated and encrypted connections to ensure the cloud services (SLA's).

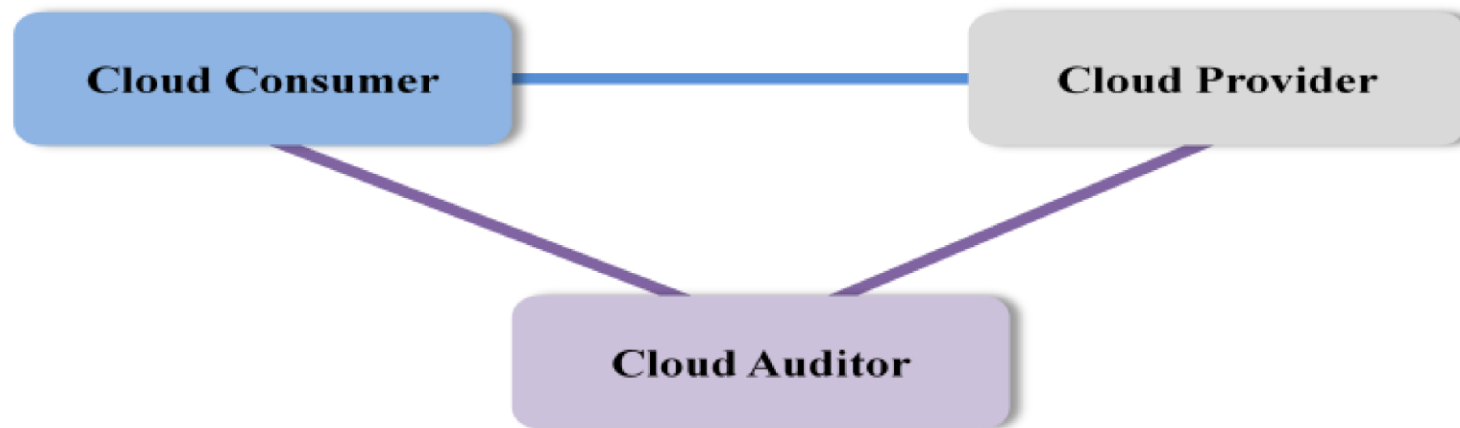


Cloud Auditor

- Audits are performed to verify conformance to standards.
- Auditor evaluates the security controls, privacy impact, performance, etc.
- Auditing is especially important for federal agencies.
- Security auditing, can make an assessment of the security controls to determine the extent to which the controls are implemented correctly, operating as intended, and producing the desired outcome. This is done by verification of the compliance with regulation and security policy.
- Privacy audit helps in Federal agencies comply with applicable privacy laws and regulations governing an individual's privacy, and to ensure confidentiality, integrity, and availability of an individual's personal information at every stage of development and operation.

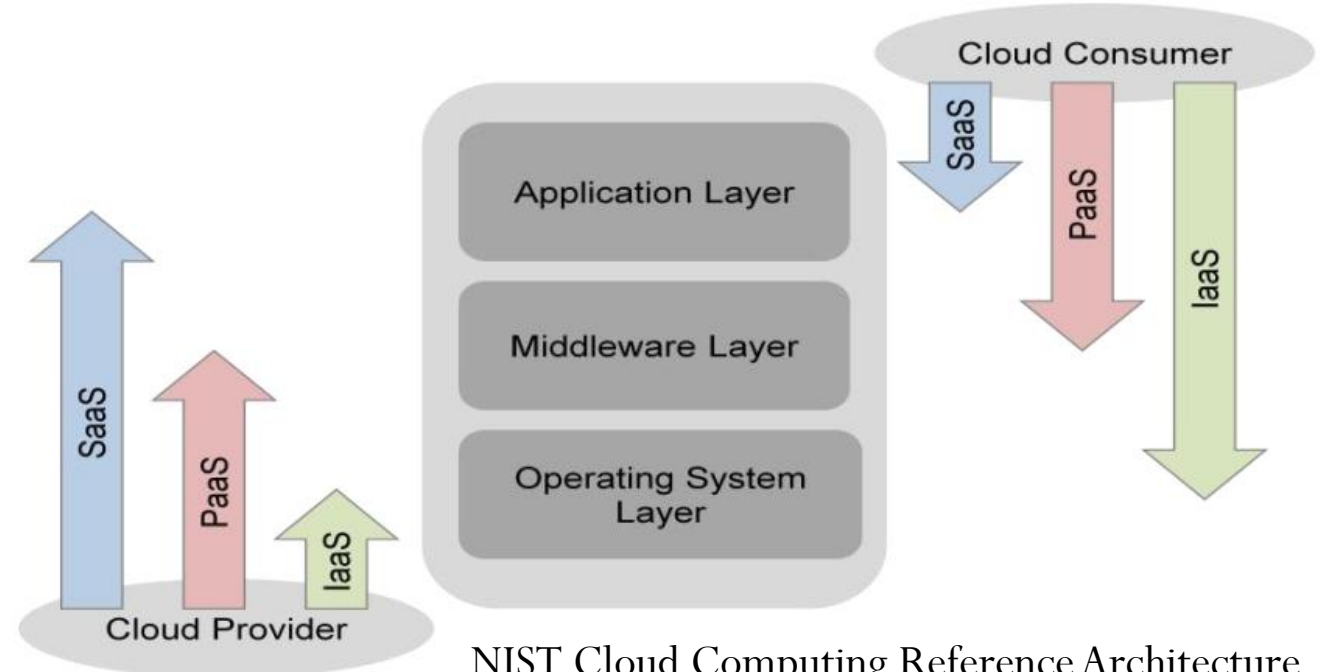
Scenarios in Cloud: 3

1. Cloud auditor conducts independent assessments for the operation and security of the cloud service.
2. The audit may involve interactions with both the Cloud Consumer and the Cloud Provider.



Scope of Control between Provider and Consumer

- Application layer are used by SaaS consumers, or installed/managed/ maintained by PaaS consumers, IaaS consumers, and SaaS providers.
- Middleware is used by PaaS consumers, installed/managed/maintained by IaaS consumers or PaaS providers. Middleware is hidden from SaaS consumers.
- IaaS layer is hidden from SaaS consumers and PaaS consumers. Consumers have freedom to choose OS to be hosted.



NIST Cloud Computing Reference Architecture

Examples of Cloud Consumers

- **SaaS Consumers:** Students and teachers using **Google Docs, Sheets, and Classroom** for assignments, collaboration, and online teaching; users accessing **LinkedIn, Facebook, Zoom, Gmail** without managing software.
- **PaaS Consumers:** Developers and data scientists using **Google Colab, Docker, Kubernetes, and AWS Elastic Beanstalk** to develop, train, and deploy AI models and cloud applications without managing servers.
- **IaaS Consumers:** AI startups and enterprises renting **NVIDIA GPU servers (GPUaaS), AWS EC2, and Google Compute Engine** to train large language models and deploy scalable cloud services.

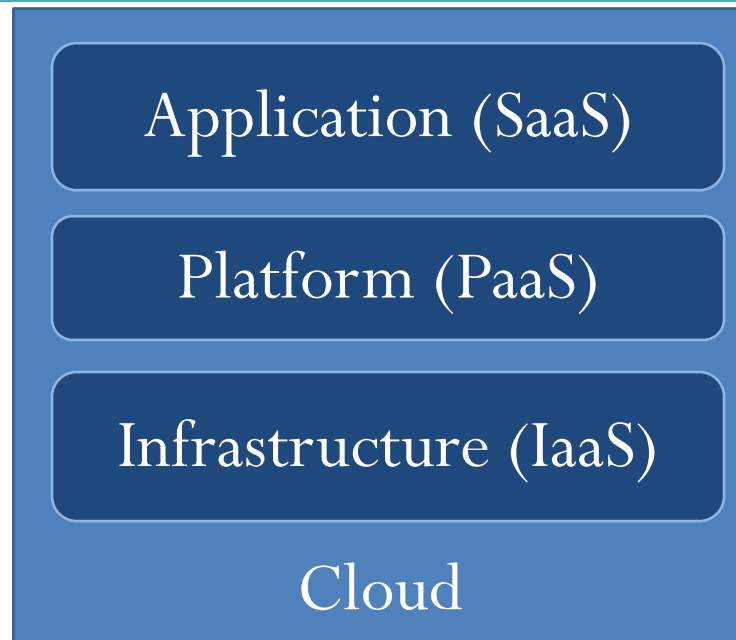
Examples of Cloud Providers

- **SaaS Providers: Google (Docs, Classroom), Microsoft (Office 365), Meta (Facebook, Instagram), Zoom** providing fully managed applications over the internet.
- **PaaS Providers: Google Colab, AWS Elastic Beanstalk, Azure App Services, Docker, Kubernetes platforms** providing development, deployment, and AI execution environments.
- **IaaS Providers: AWS, Google Cloud, Microsoft Azure, Oracle Cloud, IBM Cloud** providing virtual machines, GPU clusters, storage, and high-speed networking.
- **AI Infrastructure Providers: NVIDIA GPU Cloud, Google TPU Pods, AWS H100 clusters** providing large-scale AI compute and GPU-as-a-Service (GPUaaS).

Examples of Cloud Carriers, Brokers & Auditors

- **Cloud Carriers: Jio, Airtel, BSNL, AT&T, Verizon** providing fiber-optic, broadband, and 5G connectivity linking users to cloud data centers.
- **Cloud Brokers: Flexera (RightScale), CloudHealth, Spot.io** enabling multi-cloud management, workload optimization, and cost control.
- **Cloud Auditors: Deloitte, PwC, KPMG** auditing cloud platforms for security, compliance, data privacy, and regulatory standards (SOC2, ISO 27001, GDPR).
 - SOC (System and Organization Controls)
 - ISO (International Organization for Standardization)
 - GDPR (General Data Protection Regulation)

Cloud Services

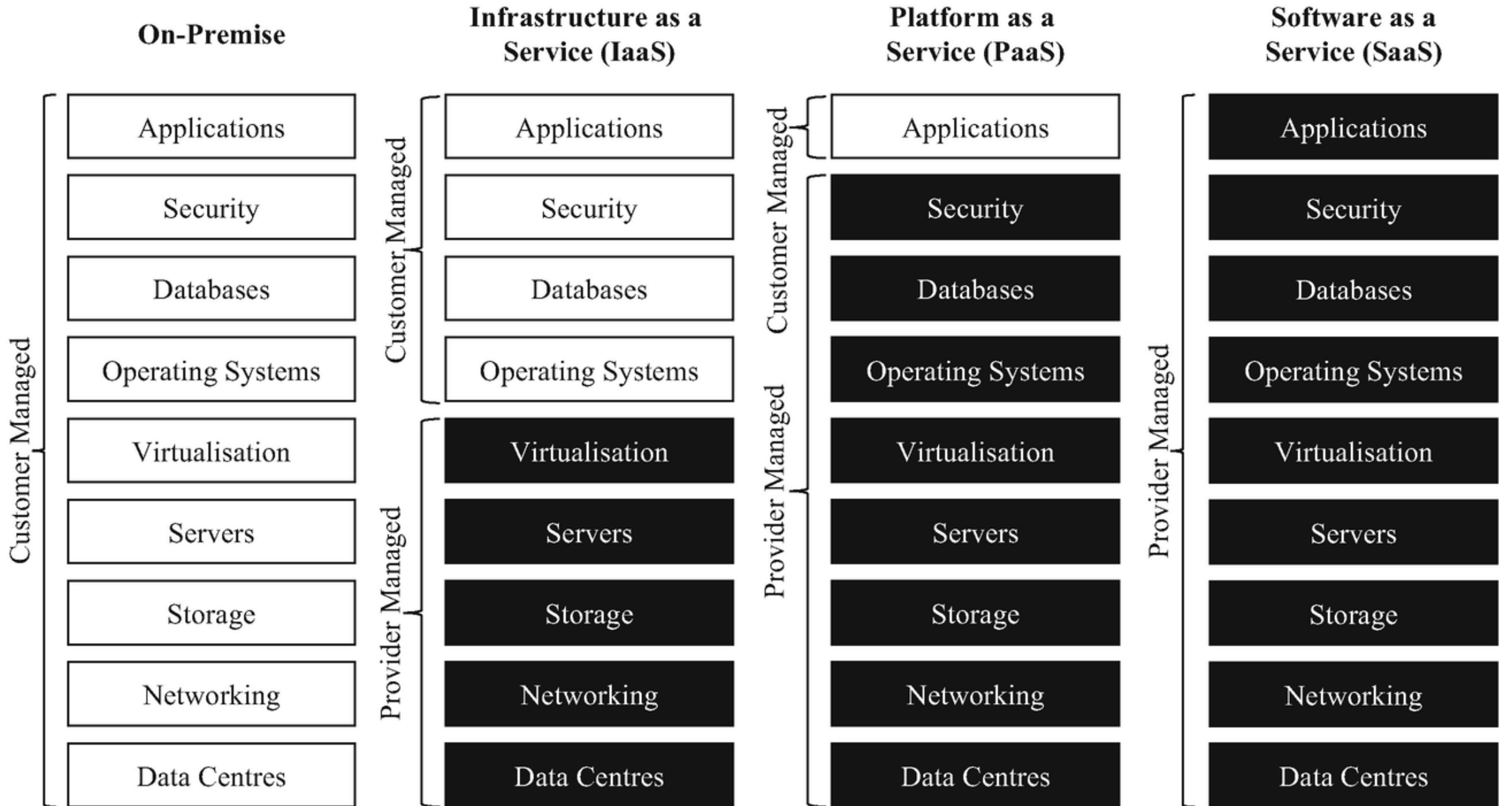


Cloud Services

- Cloud services means on demand services available to users via the network from a vendor servers.
- Cloud services are easy to use, scalable and are fully managed by a services provider.
- Services which are dynamically scale to meet the needs of its users with the help of giant hardware and software.
- A user does not require to provision or deploy resources or allocate IT staff.
- Examples: Data storage and backup solutions, Web-based e-mail, and document collaboration, database processing, and many more.

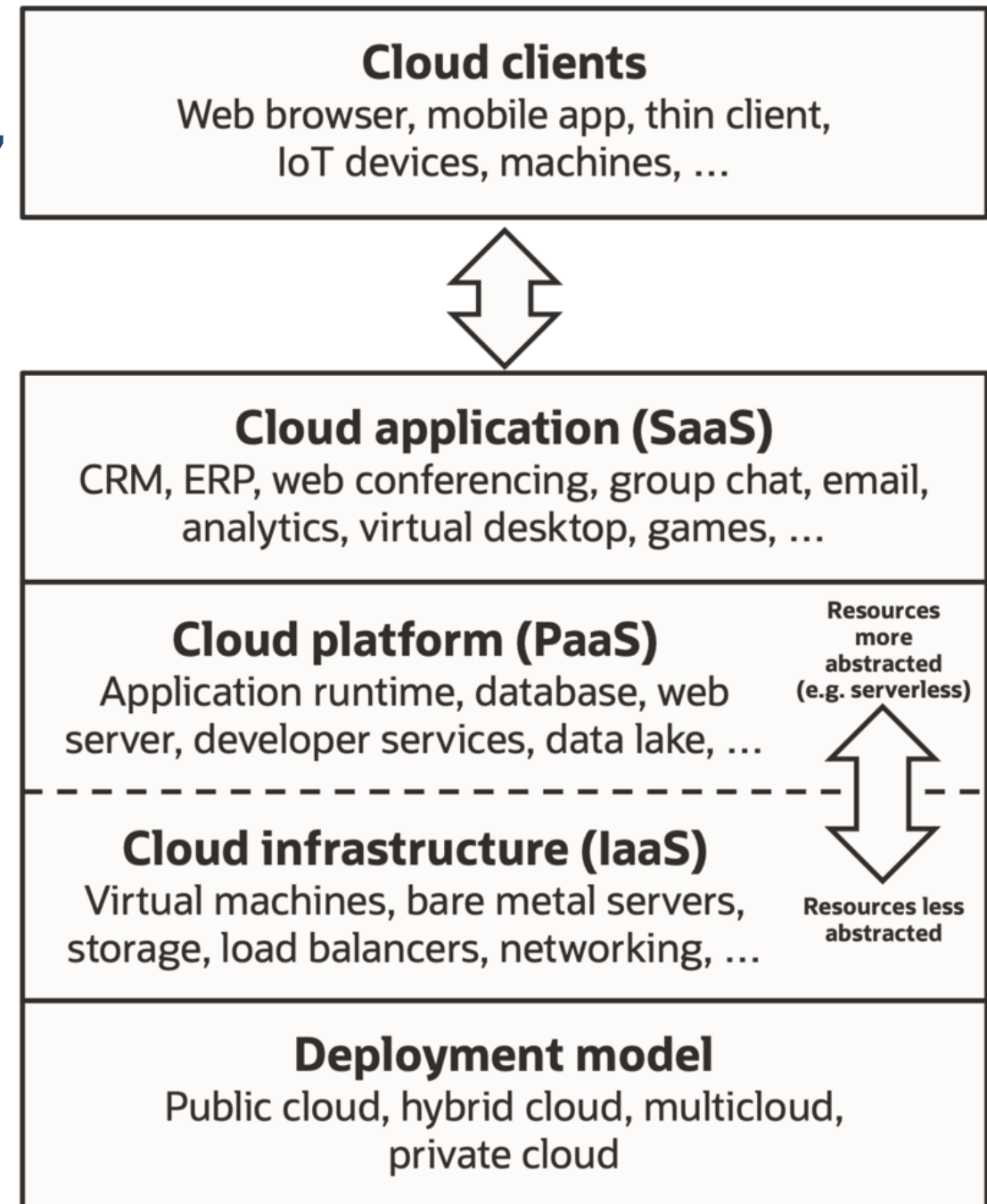
Classification and Vendors

- Infrastructure as a service (IaaS) offers hardware environment services. Kind of storage services (database or disk storage) or virtual servers.
 - Amazon EC2, Amazon S3, Rackspace Cloud Servers etc.
- Platform as a Service (PaaS) offers development (platform) environment services. PaaS offered by different vendors are generally not compatible.
 - Google Application Engine, Microsofts Azure, Salesforce etc.
- Software as a service (SaaS) offers software (applications) based on 'pay-as-you-go'
 - Salesforce in Customer Relationship Management (CRM), Gmail, Google apps (Docs, Sheets etc), and Microsoft Office online.



Different “X as a Service”

- Application-as-a-service
- Platform-as-a-service
- Integration-as-a-service
- Storage-as-a-service
- Database-as-a-service
- Information-as-a-service
- Process-as-a-service
- Security-as-a-service
- Testing-as-a-service
- Infrastructure-as-a-service
- Management/Governance-as-a-service



Example of Cloud Applications

- Cloud computing applications, are also known as Software as a Service (SaaS).
- Similar to the software's installed on computers, additionally they are available online with or without credentials (username and password).

Benefits

- Global accessibility → collaborative working with multiple users.
- Automatic Up gradation → all user on same version.
- Equal benefits to all → New developments to every user at same time.
- Safety → Wallet + Money Bank + Funds

Similarly, Data + cloud application.

Here reference of Wallet is in context of better arrangement and bank for security

- Some of the important applications: Twitter, Facebook, Google Apps (text documents, spreadsheets, slide shows, Google Docs and many more), Skype, Salesforce
- Cloud application http://en.wikipedia.org/wiki/Category:Cloud_applications

REST API

Representational State Transfer (REST)

- *Representational State Transfer* (REST) was introduced and defined in 2000 by Roy Fielding in his doctoral dissertation.
- The REST principles: "HTTP Object Model" begin and design *Uniform Resource Identifiers* (URI) standards in 1994.
- Intended to invoke an image: it is a network of Web resources (a Virtual State Machine) where the user progresses through the application by selecting resource identifiers and resource operations such as GET or POST (application state transitions),
- Resulting in the next resource's representation (the next application state) being transferred to the end user for their use.

Representational State Transfer (REST)

- WWW: "Web resources" are documents or files at URLs.
- Web 2.0: "Web Resources" are generic and abstract entities, or actions that can be identified, named, addressed, handled, or performed in many way on the internet as URI or URL.
- requests to a resource's URI:
 - a response with a payload formatted in many ways
 - e.g., HTML, XML, JSON, or some other format.
- The response given alternate resource state,
 - provide hypertext links to alternate related resources.

Representational State Transfer (REST)

- REST is a software architectural style that defines a set of constraints to be used for creating Web services.
- Web services that conform to the REST architectural style, called RESTful Web services, provide interoperability between computer systems on the internet.
- RESTful Web services allow the requesting systems to access and manipulate textual representations of Web resources by using a uniform and predefined set of stateless operations.

Representational State Transfer (REST)

HTTP Method	Operations for resource management
GET	<i>Retrieve</i> the URIs and representation of resource in the response body.
POST	<i>Create</i> a resource using the instructions in the request body. The URI of the created resource is <i>automatically assigned</i> and returned in the response <i>Location</i> header field.
PUT	<i>Replace</i> all the representations of the resources with the representation in the request body, or <i>create</i> the resource if it does not exist.
PATCH	<i>Update</i> all the representations of the resources of the resource using the instructions in the request body, or <i>may create</i> the resource if it does not exist.
DELETE	<i>Delete</i> the representations of the resources.

https://en.wikipedia.org/wiki/Representational_state_transfer

Web Oriented Architecture (WOA)

- Software architecture: extends SOA to web-based applications and sites, such as social websites and personal websites.

$WOA = SOA + WWW + REST$

- A core set of Web protocols like HTTP, HTTPS and plain XML, the only real difference between traditional SOA and the concept of WOA
- WOA advocates REST a method for HTTP as a Web service
- Enterprise WOA is a sub-style of Enterprise SOA
- Resource Oriented Architecture (ROA) use design and develop an internetworked software with resources enabled by "RESTful" interfaces.

Web Oriented Architecture (WOA)

- WOA is substyle of SOA that integrates systems and users via a web of globally linked hypermedia based on Web architecture.
- Emphasize generality of User interfaces and APIs for global network effects through five fundamental:
 - Identification of resources
 - Manipulation of resources through representations (Web resource)
 - Self-descriptive messages
 - Hypermedia as the engine of application state
 - Application neutrality

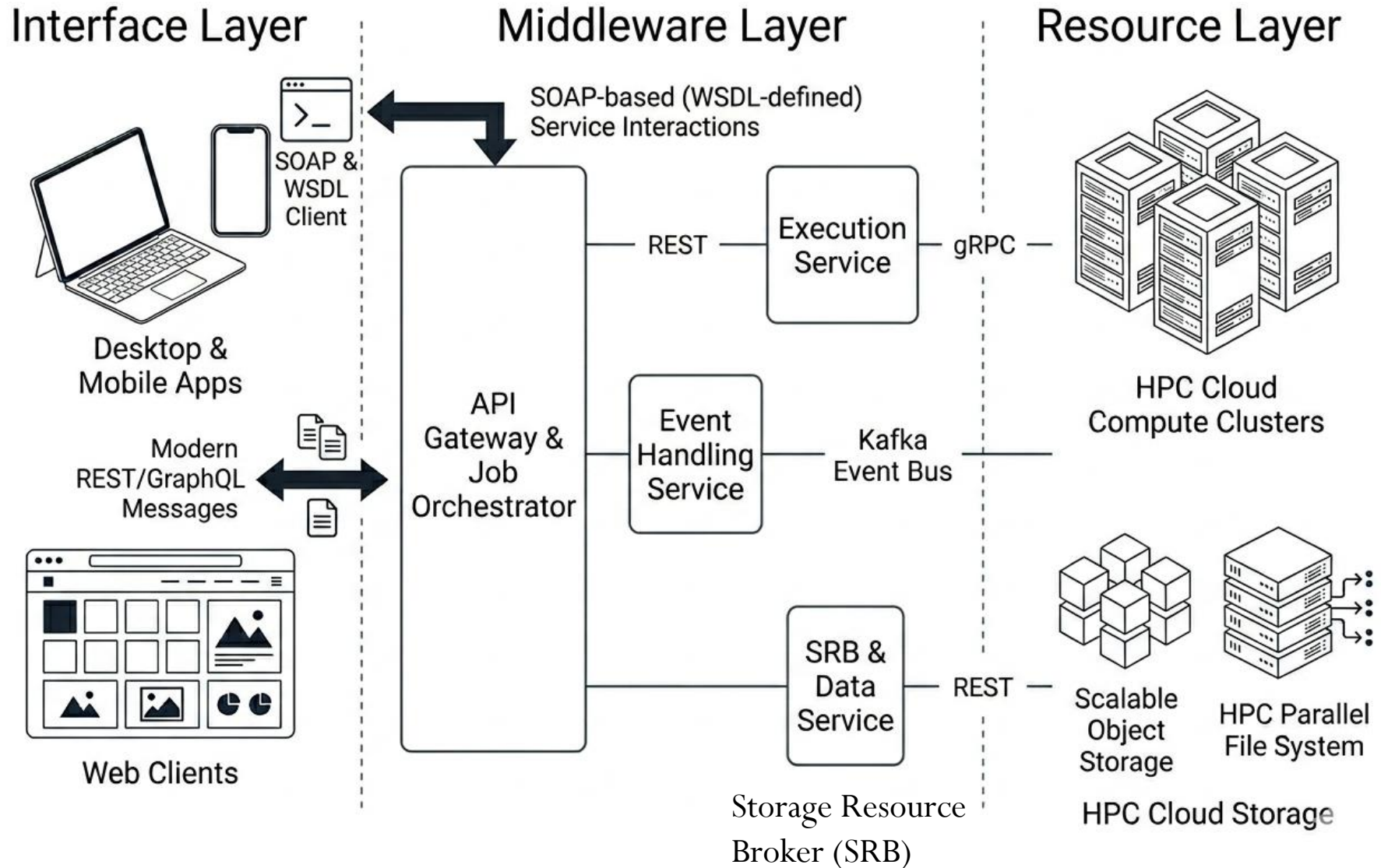
Representational State Transfer (REST)

- Stateless protocol and standard operations,
- Rest API: The operations HTTP methods available are GET, HEAD, POST, PUT, PATCH, DELETE, CONNECT, OPTIONS and TRACE.
- Ability to grow by reusing components that can be managed and updated without affecting the system.
- SOAP Web services, expose their own arbitrary sets of operations.

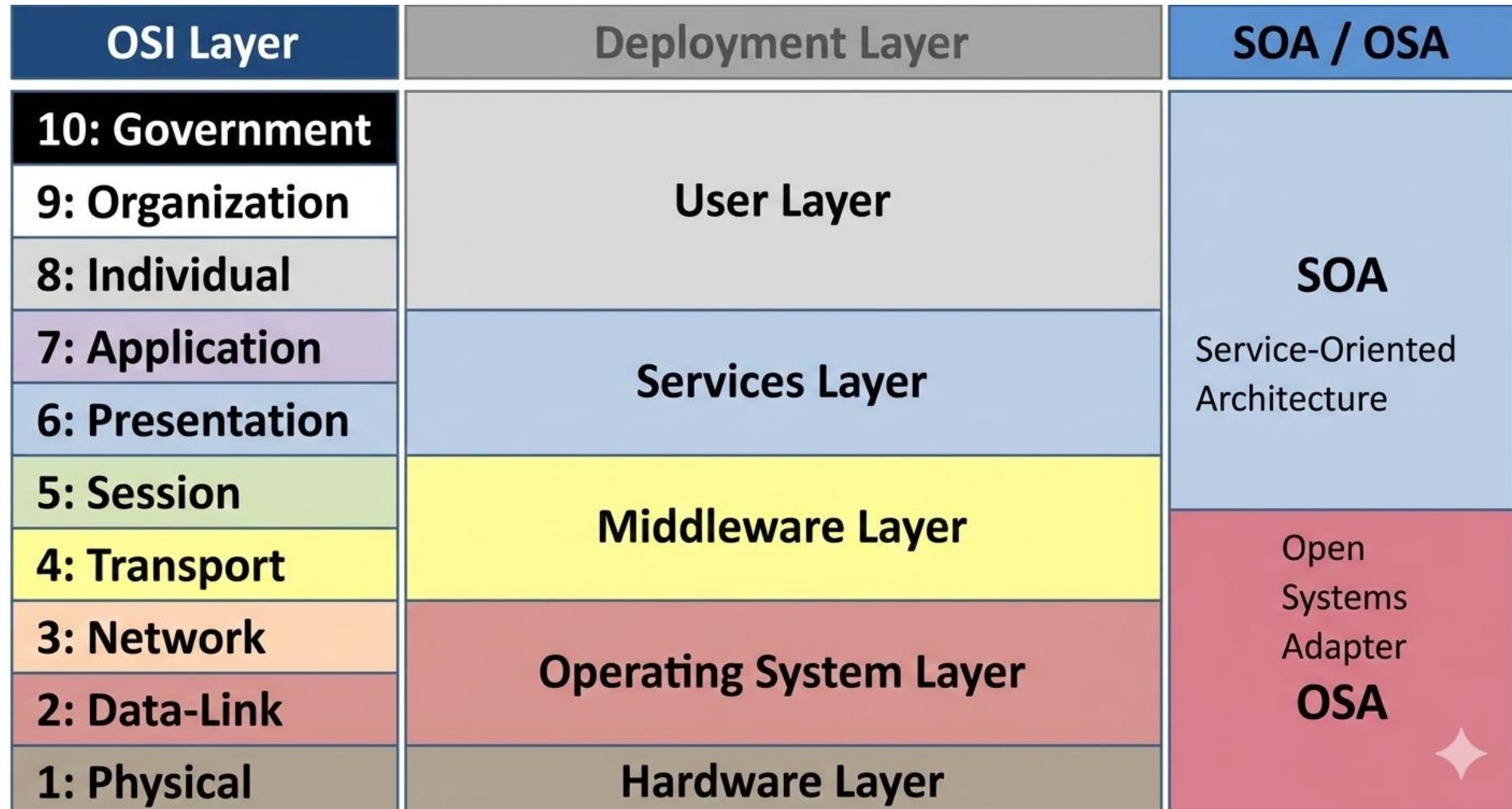
Web Services in Grid and Cloud Computing

How Services are being involved in
Grid and Cloud computing architecture.

Common Architecture



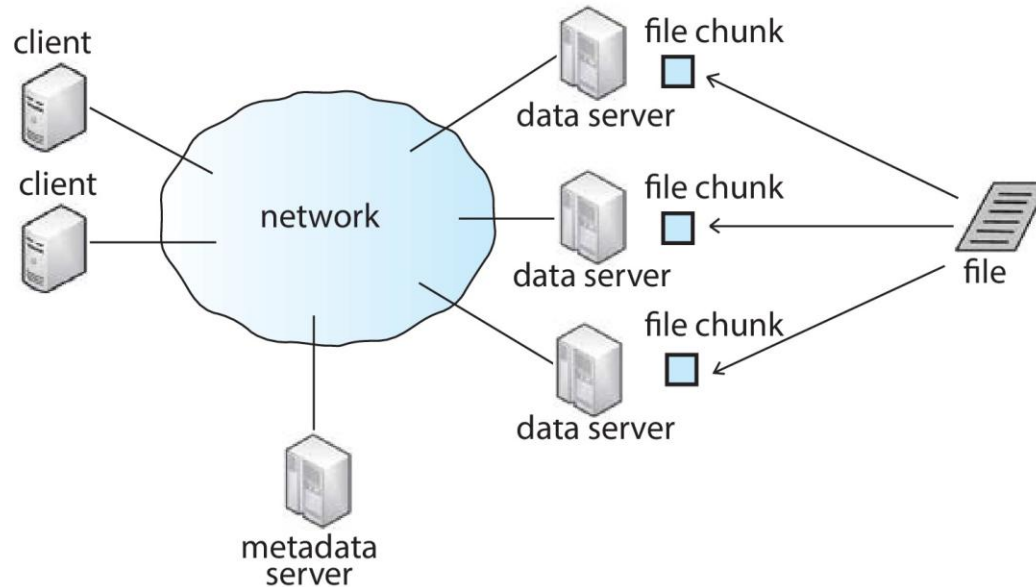
Common Architecture



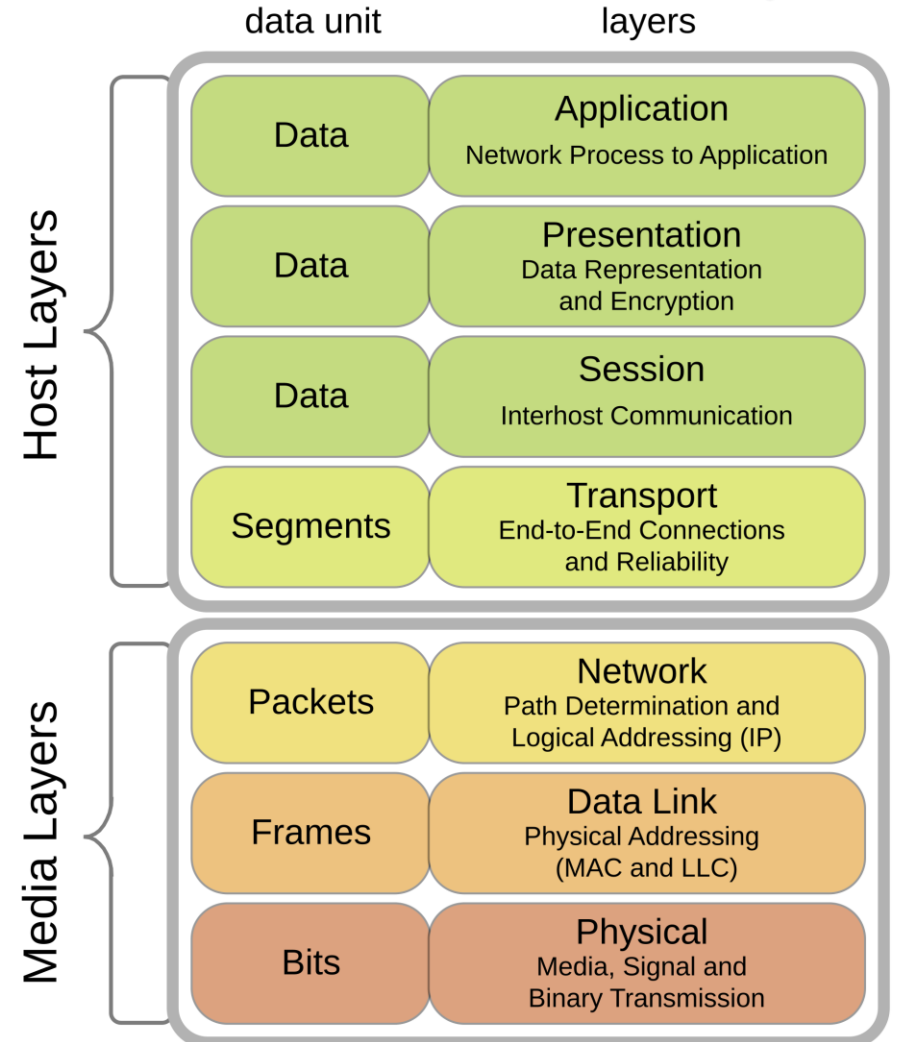
Common Architecture

Application Layer

Client-Server Layer

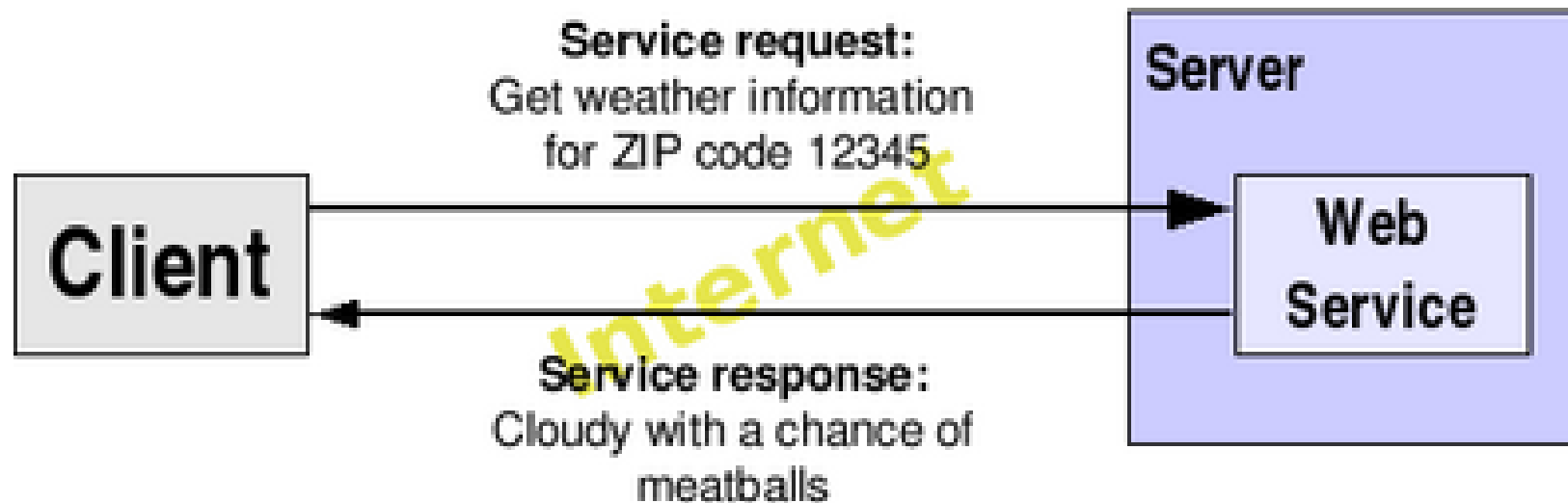


OS and Network Layers



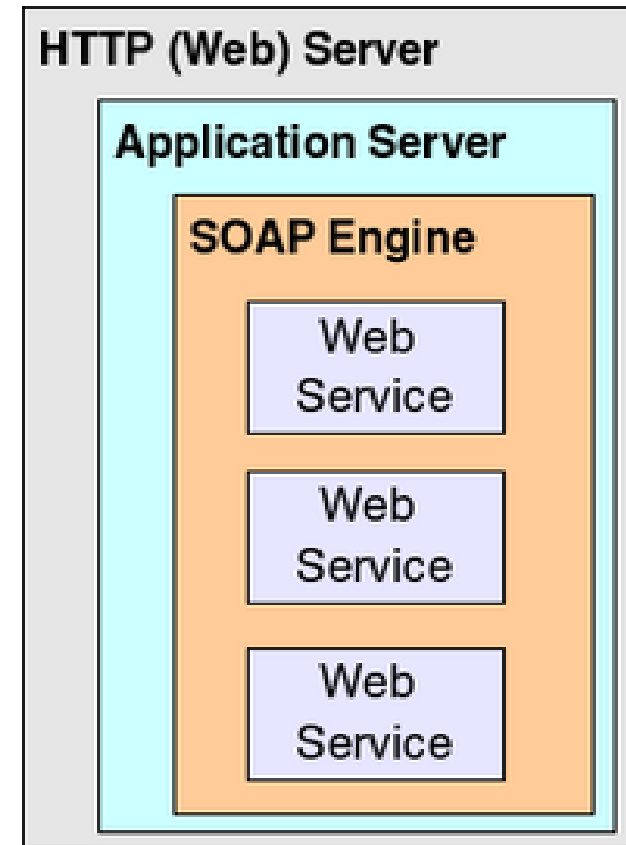
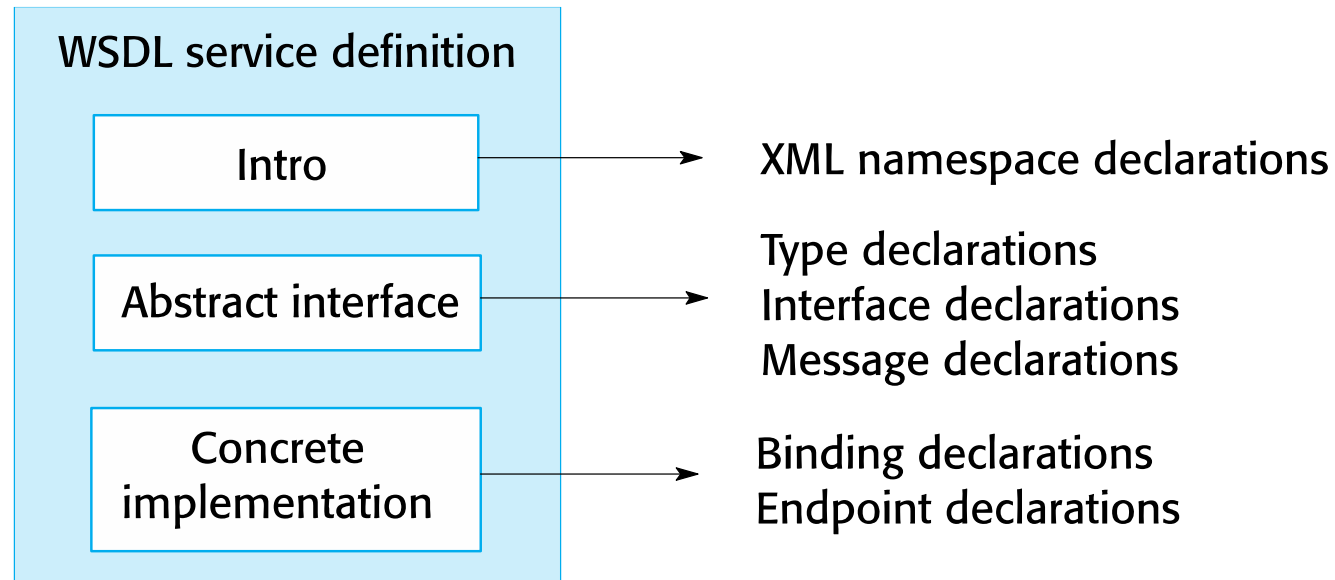
URL V/S URI

- Websites for humans, Web Services for software
- <http://webservices.mysite.com/weather/us/WeatherService>
- Web Services use HTTP as the application layer protocol to exchange SOAP messages over the network.



Web Service Invocation

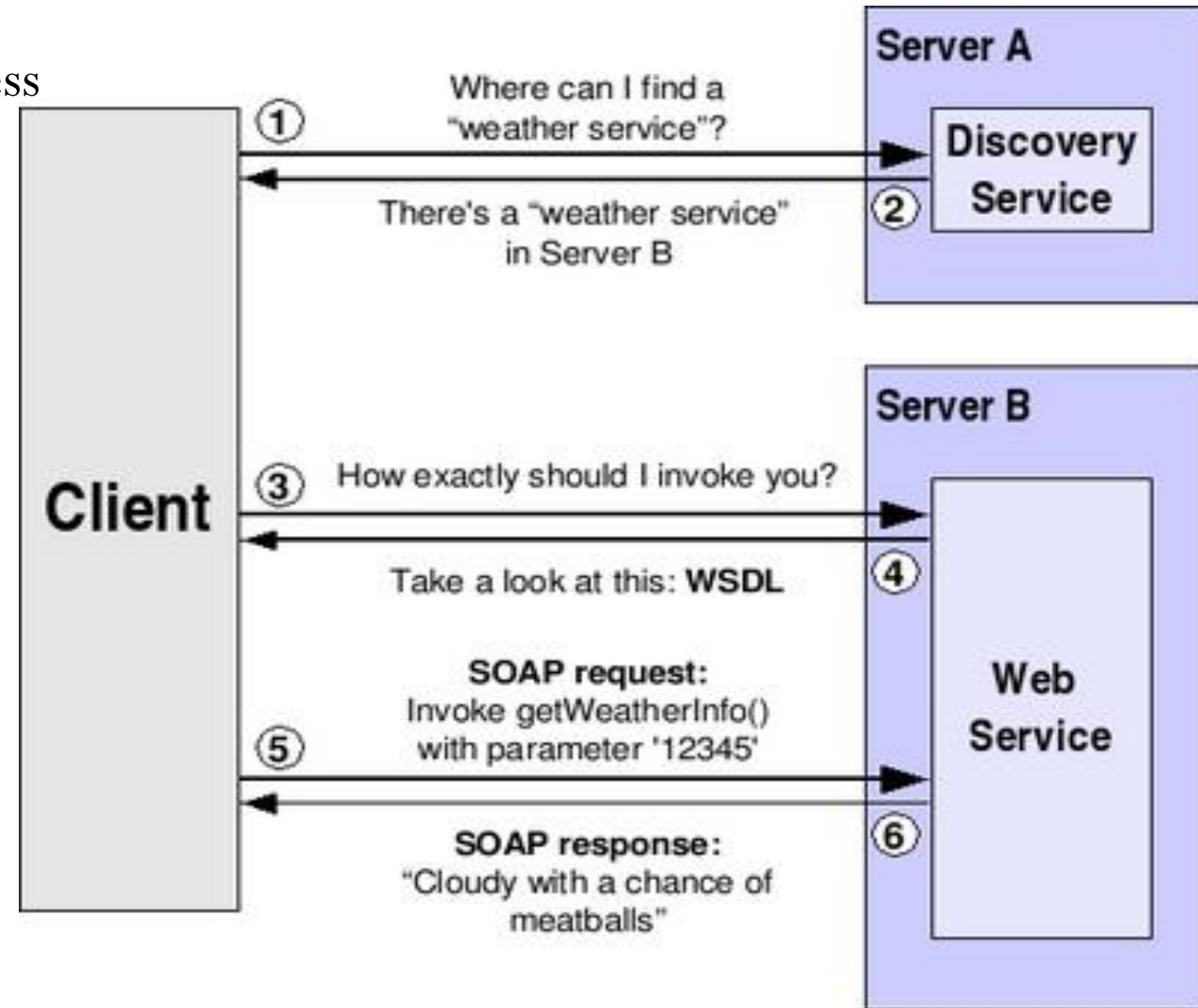
- Web service
- SOAP engine
- Application server
- HTTP Server



Web Service Invocation

Sequence of Web service invocation process

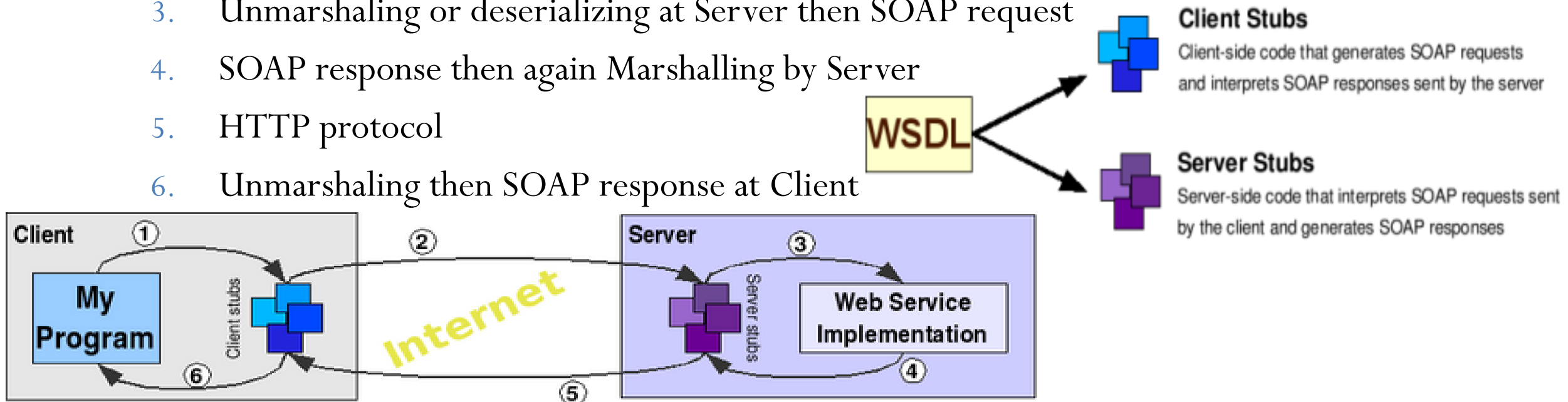
- Discovery service
- Discovery service will reply
- Web Service Descriptive Language (WSDL)
- SOAP request
- Method invocation
- SOAP response



Web Service Invocation

- A stub is an API that handles Marshalling/Unmarshalling.
- Marshalling also refers to serializing and Unmarshalling also refers to deserializing.

1. SOAP request then Marshalling or serializing by Client
2. HTTP protocol
3. Unmarshaling or deserializing at Server then SOAP request
4. SOAP response then again Marshalling by Server
5. HTTP protocol
6. Unmarshaling then SOAP response at Client



SOAP (Simple Object Access Protocol)

- SOAP is a strict, XML-based protocol for exchanging structured information between applications over networks, typically HTTP or SMTP.
- Common Use Cases
 - Financial Services: Due to high security and transactional requirements (ACID).
 - Legacy Systems: Integration with older, established, enterprise systems.
 - Telecommunications: Systems requiring strict contract enforcement.

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <m:GetStockPrice xmlns:m="http://www.example.org/stock">
      <m:StockName>IBM</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

Part of a WSDL description for a web service

Define some of the types used. Assume that the namespace prefixes 'ws' refers to the namespace URI for XML schemas and the namespace prefix associated with this definition is weathns.

```
<types>
  <xs: schema targetNamespace = "http://.../weathns"
    xmlns: weathns = "http://.../weathns" >
    <xs:element name = "PlaceAndDate" type = "pdrec" />
    <xs:element name = "MaxMinTemp" type = "mmtrec" />
    <xs: element name = "InDataFault" type = "errmess" />
```

```
  <xs: complexType name = "pdrec"
    <xs: sequence>
    <xs:element name = "town" type = "xs:string"/>
    <xs:element name = "country" type = "xs:string"/>
    <xs:element name = "day" type = "xs:date" />
  </xs:complexType>
```

Definitions of MaxMinType and InDataFault here

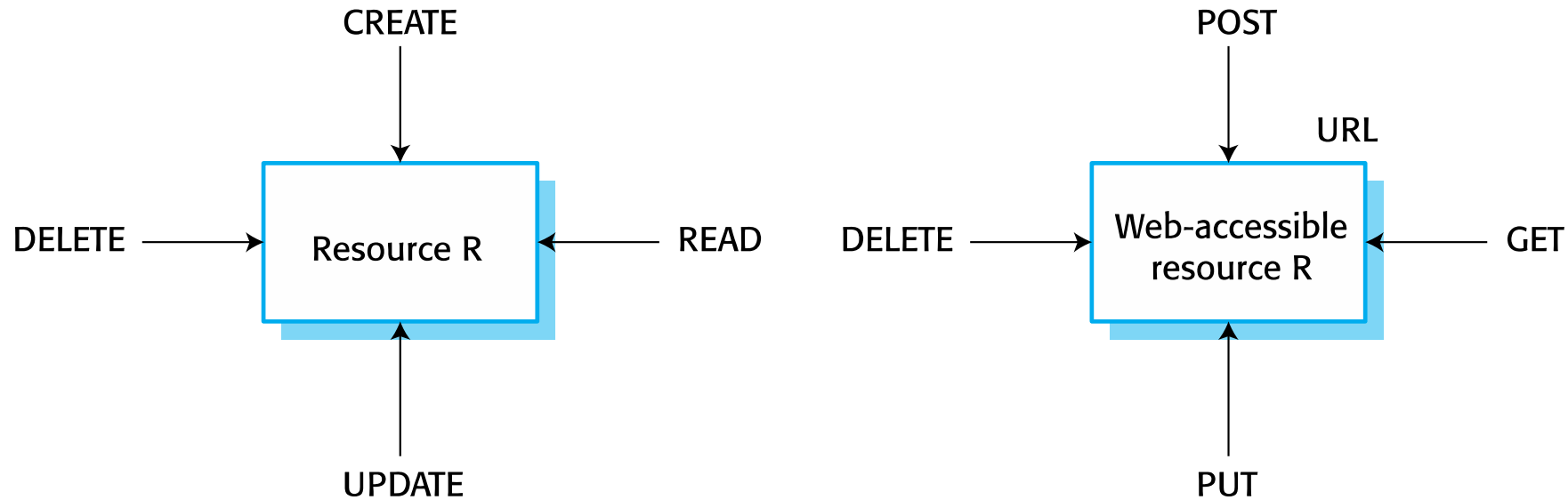
```
  </schema>
</types>
```

Now define the interface and its operations. In this case, there is only a single operation to return maximum and minimum temperatures.

```
<interface name = "weatherInfo" >
  <operation name = "getMaxMinTemps" pattern = "wsdl: in-out">
    <input messageLabel = "In" element = "weathns: PlaceAndDate" />
    <output messageLabel = "Out" element = "weathns:MaxMinTemp" />
    <outfault messageLabel = "Out" element = "weathns:InDataFault" />
  </operation>
</interface>
```

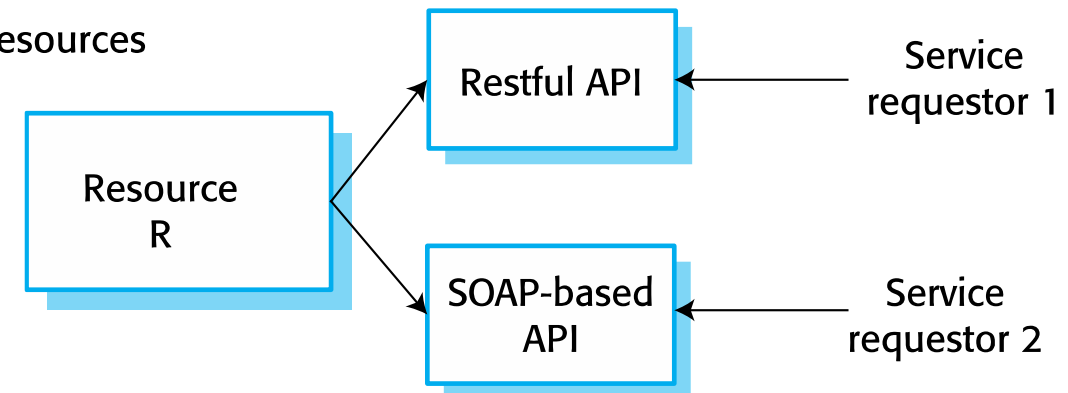
SOAP vs. REST Comparison

SOAP vs. REST Comparison



(a) General resource actions

(b) Web resources



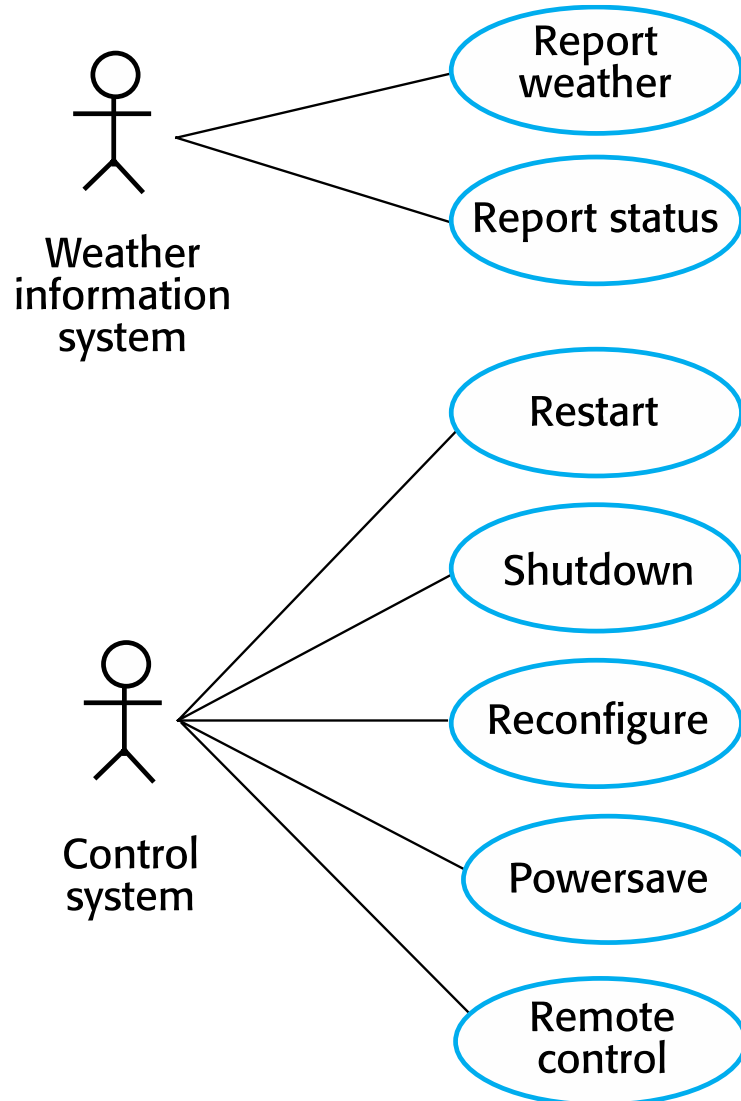
SOAP: Uses only XML, stateful, rigid, high security, slower.

REST: Uses JSON or XML, stateless, lightweight, faster, more popular for web apps.

SOAP vs. REST Comparison

Feature	SOAP (Simple Object Access Protocol)	REST (Representational State Transfer)
Design	A formal, standardized protocol with strict rules.	An architectural style with a set of guidelines.
Service Description	Requires a mandatory WSDL file to define operations and data types.	Typically uses informal documentation or optional specifications like OpenAPI; no mandatory contract file.
Data Format	Uses only XML for messaging, which is verbose.	Supports multiple formats like JSON (most common), XML, plain text, and HTML.
Transport Protocol	Can use various protocols (HTTP, SMTP, TCP, JMS).	Primarily uses only HTTP/HTTPS .
Performance	Slower due to larger XML messages and extra overhead.	Faster due to smaller messages and support for caching.
State Management	Supports both stateless and stateful operations.	Enforces statelessness (each request is independent).
Security	Built-in security standards (WS-Security, ACID compliance) for enterprise needs.	Relies on existing transport-level security (HTTPS, SSL) and tokens (OAuth, JWT).
Ease of Use	Higher learning curve and requires specialized tooling.	Simpler, uses standard HTTP methods (GET, POST, PUT, DELETE), and is easier to implement.

Weather station services



WeatherStation
identifier
reportWeather () reportStatus () powerSave (instruments) remoteControl (commands) reconfigure (commands) restart (instruments) shutdown (instruments)

WeatherData
airTemperatures groundTemperatures windSpeeds windDirections pressures rainfall
collect () summarize ()

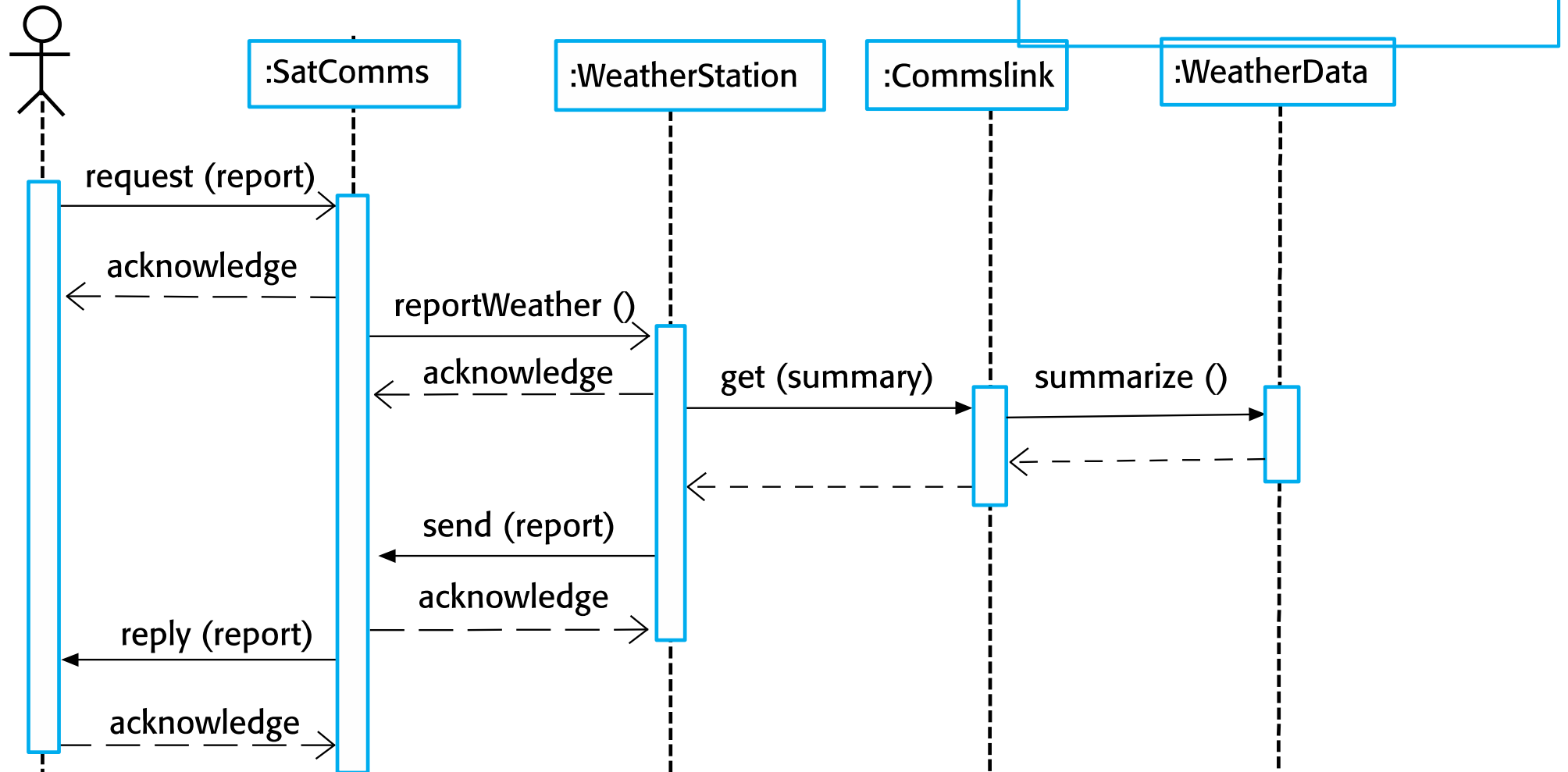
Ground thermometer
gt_Ident temperature
get () test ()

Anemometer
an_Ident windSpeed windDirection
get () test ()

Barometer
bar_Ident pressure height
get () test ()

Weather services

information system



Android App using Weather Web Service

- An Android mobile application that fetches real-time weather data from an online Weather Web Service API.
- **Architecture Flow:**
 - User → Android App → Internet → Weather API Server → JSON Response → Android UI
- **Technologies Used:**
 - Android Studio
 - Java / Kotlin
 - REST API
 - HTTP/HTTPS
 - JSON Parsing
 - Retrofit / Volley library
- **Purpose:**
 - Demonstrates how mobile applications consume cloud-based web services in real time.

How Weather Web Service Works

- **Step 1: User Input**

- User enters city name (e.g., Delhi, London).

- **Step 2: API Request**

- Android app sends HTTP request to weather server:
- `https://api.weather.com/data?city=Delhi&apikey=XXXX`

- **Step 3: Server Processing**

- Cloud server processes request using:
- Databases
- Weather sensors
- Forecast models

How Weather Web Service Works

- **Step 4: JSON Response**

- Server returns structured JSON data:
- ```
{
 "city": "Delhi",
 "temperature": "32°C",
 "humidity": "70%",
 "condition": "Cloudy"
}
```

- **Step 5: UI Display**

- Android app parses JSON and displays:
- Temperature
- Humidity
- Weather condition
- Icons

# Cloud & Web Service Concepts Demonstrated

- **Cloud Concepts:**
  - SaaS: Weather service accessible via internet
  - PaaS: API hosted on cloud platform
  - IaaS: Cloud servers storing and processing weather data
- **Web Service Concepts:**
  - RESTful API
  - Stateless communication
  - Client–Server architecture
  - JSON data format
  - API key authentication
- **Real-World Learning Outcomes:**
  - Understand API integration in Android
  - Learn HTTP request/response lifecycle
  - Practice cloud-based mobile app development
  - Build scalable real-time applications

## OSI-Layer aligned Users (Layers 8, 9, 10)

Modern Client Interaction  
(WSDL/SOAP/REST)



Desktop &  
Mobile Apps

Contextual  
(Geo-tagged)  
Service Messages



Web Clients

WSDL-  
defined,  
RESTful,  
GraphQL

## Deployment Layer

User Layer

Services Layer

Middleware Layer

Operating System Layer

Hardware Layer

## Service & Resource Layer

**SOA**  
Service-  
Oriented  
Architecture

**Service Request:**  
Get Weather Info  
for Long/Lat  
coordinates

Lat/  Long

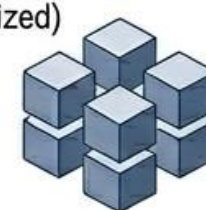


Service Gateway &  
Job Orchestrator

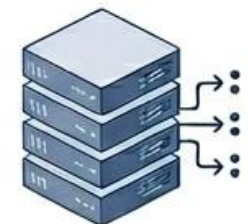
**Service Request:**  
Get Local Search Info  
for ZIP code 

REST

**Service Response:**  
Localized Weather/Search  
(Personalized)



Scalable  
Object  
Storage



HPC Parallel  
File System

HPC Cloud Compute Clusters

# Evolution of the Web

## Web 1.0 → Web 2.0 → Web 3.0

From Read-Only to Read-Write-Own



# Web 1.0 – The Read-Only Web (1990–2005)

- Web 1.0 was the first generation of the web focused on static information sharing.
- **Characteristics**
  - Static HTML pages
  - One-way communication
  - No user-generated content
  - Centralized servers and hosting
  - Minimal interactivity
- **Technologies:** HTML, HTTP, Basic CSS, Web servers
- **Examples:** Early Yahoo directory, Static personal websites, Online encyclopedias
- **Model:** User → Website (Read Only)

# Web 2.0 – The Read-Write Web (2005–Present)

- Web 2.0 introduced user interaction, social media, and dynamic web applications.
- **Characteristics**
  - Dynamic & interactive websites
  - User-generated content & social media
  - Cloud-based platforms
  - Centralized data control
  - Examples: Facebook, YouTube, Google Docs
- **Technologies:** JavaScript, AJAX, REST APIs, Cloud computing, Mobile apps
- **Examples:** Facebook, YouTube, Wikipedia, Instagram, Google Docs
- **Model:** User ↔ Platform (Read + Write)
  - ⚠ Data controlled by centralized companies

# Web 2.0

## Features

- Web 2 based Homepage
- Search
- Really Simple Syndication (RSS)
- Chats, Sharing
- Personalized web sites,
- Mashups
- POD Casting, Video Casting, YouTube
- Focus on community, online collaboration, tagging

# RDF Site Summary (RSS) feed

- aka Really Simple Syndication (RSS) feed
- Resource Description Framework (RDF): a general framework for representing interconnected data on the web.
- RSS feed is an XML language for syndicating subscribed contents on the web.
- Registers with an organization on web, who creates an RSS feed.
- User is automatically updated with the RSS feed (notification of new items)
- Information is received quickly at a location.

# Mashup

- A mashup, is a phenomenon to create new services on web, with the combination of presentation, data, or functionality from two or more resources.
- What are the benefits?
  - Innovation, new business insights, increase agility, speed up the development, reduce development costs, easy & fast integration, widgets are readily available, reuse software, immediate benefit, less cost, rich ecosystem, may not be original raw source data, produce enriched results of data & services
- Mashup is divided into three layers with the following technologies.
- **Presentation:** [HTML](#), [XHTML](#), [CSS](#), [Javascript](#), [Ajax](#), [API](#).
- **Web Services:** are [XMLHttpRequest](#), [XML-RPC](#), [JSON-RPC](#), [SOAP](#), [REST](#).
- **Data:** Sending, storing and receiving. The technologies used are [XML](#), [JSON](#), [KML](#).

# Web 3.0 – Read-Write-Own Web

- Web 3.0 (Web3) represents the next evolution of the internet
  - shift from Read-Write to Read-Write-Own.
  - a decentralized internet where users control their data, identity, and digital assets.
- Key Technologies of Web 3.0
  - **Artificial Intelligence (AI):** Generative AI, Agentic AI, Discriminative AI etc.
    - **Large Language Models (LLMs)** based chatbots: ChatGPT, Gemini, etc
  - **Decentralized Storage** (example: InterPlanetary File System (IPFS))
  - **Edge Computing:** autonomous vehicles, industrial robots, and smart city traffic
  - **Blockchain & Smart Contracts:** Ethereum, Hyperledger Fabric, etc
    - Non-fungible tokens (NFTs), Decentralized Finance (DeFi), Decentralized Autonomous Organization (DAO)
  - **AR/VR & Spatial Computing:** Web-based Gaming, E-Commerce / Retail, Advertising/Marketing, Educational content, Remote assistance
  - **Cryptographic Wallets:** Unified Payments Interface (UPI)

# Web 3.0 (2014 onwards)

## Core Characteristics of Web 3.0

- **Decentralization:** Data distributed across peer-to-peer networks.
- **Trustless & Permissionless:**
  - Users own data & digital assets.
  - No intermediaries required.
  - Anyone can join without approval.
- **Peer-to-peer architectures:** WhatsApp
- **Semantic Web:** Machines understand context and meaning of information.
- **Ubiquity:** Internet integrated with IoT, 5G, and smart devices.

# Web 3.0 & Cloud Convergence

- **Conceptual Shift: From "Centralized" to "Decentralized"**
  - **Web 2.0 (The Cloud Era):** Dominated by centralized cloud providers (AWS, Azure, Google). Data is stored in silos; users trade privacy for services.
  - **Web 3.0 (The Ownership Era):** A "Read-Write-Own" model where data is distributed across a peer-to-peer network using **Blockchain** and **Edge Computing**.
- **Core Pillars for Cloud & AI Architecture**
  - **Decentralized Infrastructure:** Replaces central servers with distributed nodes. No single point of failure (Immutability).
  - **AI-Driven Intelligence:** Moves beyond keyword searches to **Semantic Web** processing, where AI understands context to deliver hyper-personalized user experiences.
  - **Trustless Transactions: Smart Contracts** automate logic and payments without intermediaries, reducing latency in cloud-based service delivery.



# WEB 1.0

(The Read-Only Web)  
(c. 1990s)



Static HTML pages, content consumption, limited user interaction, centralized servers.



STATIC PAGES, HTML, EMAIL, BROWSING

# WEB 2.0

(The Read-Write Web)  
(c. 2000s-2010s)



User-Generated Content, Social Media, Online Communities, Collaboration, Mobile Access.



SOCIAL MEDIA, MOBILE APP, AJAX /JS, CLOUD, BLOGS

# WEB 3.0

(The Semantic/Decentralized Web)  
(c. 2020s+)



Decentralized, User-Owned Data, Blockchain, Cryptocurrencies, Semantic Web (AI/ML), Interoperability



BLOCKCHAIN, CRYPTO, NFTs, DAO, SMART CONTRACTS, DECENTRALIZED IDENTITY

# Web 1.0 vs Web 2.0 vs Web 3.0 (Comparison Table)

| Feature      | Web 1.0       | Web 2.0               | Web 3.0                         |
|--------------|---------------|-----------------------|---------------------------------|
| Period       | 1990–2005     | 2005–Present          | 2014–Future                     |
| Main Idea    | Information   | Interaction           | Ownership                       |
| User Role    | Reader        | Creator               | Owner                           |
| Content      | Static        | Dynamic               | Decentralized Apps              |
| Data Control | Website owner | Centralized platforms | User-controlled (Blockchain)    |
| Architecture | Client–Server | Client–Server + Cloud | Peer-to-Peer                    |
| Trust        | Trust website | Trust company         | Trust code                      |
| Monetization | Ads           | Targeted ads          | Tokens, NFTs, DeFi              |
| Technologies | HTML          | JS, APIs, Cloud       | Blockchain, AI, Smart Contracts |

# Evolution of Web

- Web 1.0 → Information Access (Read)
- Web 2.0 → Social Interaction (Read + Write)
- Web 3.0 → Digital Ownership (Read + Write + Own)
- **Synergies:** AI in the Web 3.0 Cloud
- **Privacy-Preserving AI:** Training models on encrypted, decentralized data sets without exposing sensitive user information.
- **Generative Web:** Using AI (LLMs) to dynamically create 3D environments and assets for the **Metaverse** in real-time.
- **Major Shift:** Centralized Internet → Social → Decentralized Internet
  - From: User as Consumer
  - To: User as Owner & Stakeholder

# Evolution of Web

- The evolution of the web shows a clear transformation:
  - From static content
  - To interactive platforms
  - To decentralized ownership models
- Web 3.0 aims to create:
  - Transparent systems
  - Secure digital identities
  - Trustless transactions
  - User-controlled ecosystems
- The future internet will combine:  
AI + Blockchain + Cloud + IoT + Edge Computing



# References

- NIST Cloud Computing Reference Architecture
- Borja Sotomayor, “The Globus Toolkit 4 Programmer's Tutorial,”
- [https://en.wikipedia.org/wiki/Representational\\_state\\_transfer](https://en.wikipedia.org/wiki/Representational_state_transfer)
- [https://en.wikipedia.org/wiki/Web-oriented\\_architecture](https://en.wikipedia.org/wiki/Web-oriented_architecture)
- Avi Silberschatz, Peter Baer Galvin, and Greg Gagne. Operating System Concepts (Tenth Edition). <https://www.os-book.com/OS10/slide-dir/index.html>
- Software Engineering, Ian Sommerville, 9th Pearson 2011.
- Software Engineering, Ian Sommerville, 10th Pearson 2018.
- <https://software-engineering-book.com/slides/>

תודה רבה

Hebrew

Ευχαριστώ

Greek

Спасибо

Russian

Danke

German

धन्यवादः

Sanskrit

நன்றி

Tamil

شكراً

Arabic

Merci

French

ধন্যবাদ

Bangla

ಧನ್ಯವಾದಗಳು

Kannada

Thank You

English

നന്ദി

Malayalam

Grazie

Italian

ధన్యవాదాలు

Telugu

આભાર

Gujarati

多謝

Traditional Chinese

Gracias

Spanish

ਧੰਨਵਾਦ

Punjabi

धन्यवाद

Hindi & Marathi

多谢

Simplified Chinese

<https://sites.google.com/site/animeshchaturvedi07>

Obrigado

Portuguese

ありがとうございました

Japanese

ขอบคุณ

Thai

감사합니다

Korean